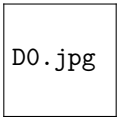
Lecture 24: CTEs and Recursive Queries

Dr. Innocent Nyalala
IIT Madras Zanzibar



School of Engineering and Science
IIT Madras Zanzibar

4 May 2026

Lecture Outline

Today's Road Map

1. The problem CTEs solve
2. CTE syntax & simple examples
3. Multiple / chained CTEs
4. Recursive CTEs — motivation
5. Recursive CTE syntax
6. Step-by-step execution trace
7. Termination & safety

Schema Used Today

```
Employee(
  EmpID, Name,
  ManagerID,
  Dept, Salary)
```

Self-referencing ⇒ org chart hierarchy

The Problem CTEs Solve

Without CTE — Hard to Read

```
SELECT Name, Salary
FROM Employee
WHERE Salary > (
  SELECT AVG(Salary)
  FROM (
    SELECT Dept, AVG(Salary) AS Salary
    FROM Employee
    GROUP BY Dept
  ) AS DeptAvg
);
```

One giant nested formula — impossible to debug!

With CTE — Crystal Clear

```
WITH DeptAvg AS (
  SELECT Dept, AVG(Salary) AS AvgSal
  FROM Employee GROUP BY Dept
),
OverallAvg AS (
  SELECT AVG(AvgSal) AS Threshold
  FROM DeptAvg
)
SELECT Name, Salary
FROM Employee, OverallAvg
WHERE Salary > Threshold;
```

Named steps — read like a recipe!

CTE = giving a name to an intermediate result

CTE Syntax

Step 1 — Define the CTE

```
WITH cte_name AS (
  SELECT col1, col2 FROM table WHERE cond)
```

feeds into

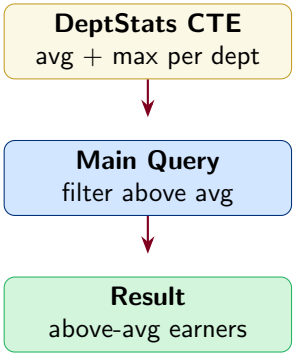
Step 2 — Use It Like a Table

```
SELECT * FROM cte_name WHERE ...
```

- Defined only for **one query**
- Can reference CTE multiple times
- Improves readability greatly
- No performance penalty (usually)

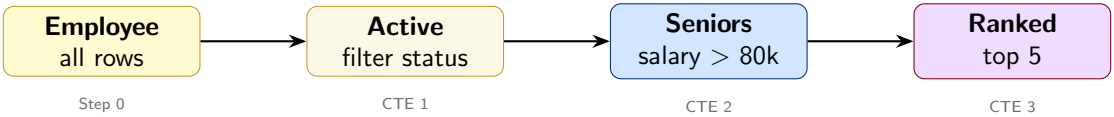
Simple CTE Example — Top Earners per Dept

```sql
-- Step 1: compute dept avg
WITH DeptStats AS (
  SELECT Dept,
    AVG(Salary) AS AvgSal,
    MAX(Salary) AS MaxSal
  FROM Employee
  GROUP BY Dept
)
-- Step 2: find above-average
   earners
SELECT E.Name, E.Salary, D.AvgSal
FROM Employee E
JOIN DeptStats D
  ON E.Dept = D.Dept
WHERE E.Salary > D.AvgSal
ORDER BY E.Salary DESC;
```



Multiple CTEs — Data Pipeline

```
WITH
  Active AS (SELECT * FROM Employee WHERE Status = 'Active'),
  Seniors AS (SELECT * FROM Active WHERE Salary > 80000),
  Ranked  AS (SELECT *, RANK() OVER (ORDER BY Salary DESC) AS Rnk
              FROM Seniors)
SELECT Name, Salary, Rnk FROM Ranked WHERE Rnk <= 5;
```



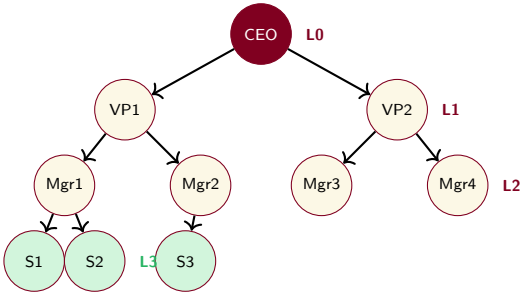
- Later CTEs **can reference earlier** CTEs
- Each stage has a **meaningful name**

What is a Recursive CTE?

The Question

"Find **all employees** who report to the CEO — directly or through any number of levels."

- Simple JOIN only finds **level 1**
- Need to follow chain **until no more managers**
- Solution: **Recursive CTE**



Recursive CTE Syntax

Anchor Member — base case (non-recursive SELECT)

UNION ALL

Recursive Member — references the CTE itself

```
WITH RECURSIVE OrgTree AS (
  -- Anchor: start at CEO (no manager)
  SELECT EmpID, Name, ManagerID, 0 AS Level
  FROM Employee WHERE ManagerID IS NULL

  UNION ALL

  -- Recursive: join to find direct reports
  SELECT E.EmpID, E.Name, E.ManagerID, T.Level + 1
  FROM Employee E
  JOIN OrgTree T ON E.ManagerID = T.EmpID
```

- Each pass finds **next level**
- Stops when join returns **no rows**

- All iterations **UNION'd together**
- Final result = entire org tree

Termination — Never Forget the Base Case!

Danger: Infinite Loop

If every row always finds a match, the recursion **never stops**.

Example: circular ManagerID references
`A` → `B` → `C` → `A` → ...

SQL Server Safety Net

```
OPTION (MAXRECURSION 100)
-- Default = 100 levels
-- 0 = unlimited (dangerous!)
```

Safe — Base case terminates chain

Unsafe — No termination ⇒ infinite loop

↓ SQL Server kills it

MAXRECURSION
last line of defence

Rule: Always have a `WHERE` or structural end condition in the recursive member

Think-Pair-Share (5 min)

Problem

Using the `Employee(EmpID, Name, ManagerID, Dept, Salary)` table:

1. Write a **recursive CTE** `Ancestors` that, given an `EmpID` as input, finds **all managers above that employee** up to the CEO.
2. Add a **Level** column showing how many hops away each ancestor is.

Hint — Anchor Member

```
SELECT EmpID, Name,
       ManagerID, 0 AS Level
FROM Employee
WHERE EmpID = :target_id
```

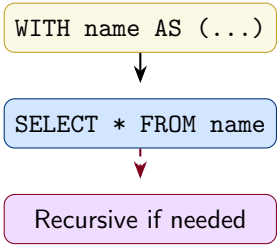
Hint — Recursive Member

```
SELECT E.EmpID, E.Name,
       E.ManagerID,
       A.Level + 1
FROM Employee E
JOIN Ancestors A
  ON E.EmpID = A.ManagerID
```

Summary

Key Takeaways

- CTEs **name intermediate results**
- Multiple CTEs = **data pipeline**
- Recursive CTEs handle **hierarchies**
- **Anchor** seeds, recursive **extends**
- Always ensure **termination**



Next: L25 — Window Functions I (ROW_NUMBER, RANK, DENSE_RANK)